

CSIS 2300 Introduction to Algorithms

Spring 2026 – Homework 2 – Asymptotic analysis - **Solutions**

(1) For each of the following algorithms, indicate (i) a natural input size metric (i.e. What quantity is n for this algorithm?), (ii) the *critical operation* (the one we should be most worried about counting in a runtime analysis), and (iii) the worst-case time complexity of the algorithm in big- Θ notation.

(a) computing the sum of a list of integers

(i) The number of integers in the list

(ii) Addition of two integers

(iii) $\Theta(n)$

(b) finding the largest element in a list of integers

(i) The number of integers in the list

(ii) Comparison of two integers

(iii) $\Theta(n)$

(c) sieve of Eratosthenes (discussed in Lecture 1)

(i) The largest number we are interested in (Or: the number of bits it has)

(ii) Addition of two integers (the main loop just looks at $a, 2a, 3a, \dots$)

(iii) This turns out to be much more difficult than I realized. Assuming n is the largest value and you optimize the algorithm, the runtime is $O(n \log \log n)$. But TAs should allow any remotely reasonable answer.

(d) standard pencil-and-paper algorithm for multiplying two integers

(i) The number of digits in each value (call them m and n)

(ii) Single-digit multiplications

(iii) $\Theta(mn)$

Note: Accept any answer that deals with digits/bits of inputs, even if it doesn't split into two parts. (We would then get $\Theta(n^2)$.)

(2) Let $p(n)$ be an arbitrary polynomial of degree k with non-negative coefficients – that is, $p(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$ with all $a_i \geq 0$. Prove that $p(n) = \Theta(n^k)$.

A correct answer here provides three constants: c_1, c_2 , and n_0 . (Important: It must be the same n_0 for both bounds.) The slides use c_1 as the upper bound constant and c_2 as the lower bound, and I will do that here, but it's fine if these are reversed so long as the logic is right. For c_2 , a_k is an obvious choice – since all of the a_i are non-negative, we are guaranteed $p(n) \geq a_k n^k$ for any $n \geq 0$. (This is not the only answer, but it's the easiest.)

For the upper bound, we should “overkill” either c_1 or n_0 . One easy possibility is to set $c_1 = a_k + a_{k-1} + \dots + a_1 + a_0$. When n is positive, we are guaranteed $n^k \geq n^j$ for all $j < k$, and so $p(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 \leq a_k n^k + a_{k-1} n^k + \dots + a_1 n^k + a_0 n^k = c_1 n^k$. This allows us to select $n_0 = 1$, since it works for any positive n .

(3) Let $f(n) = a^n$ and $g(n) = b^n$, with $a > b > 1$. Prove that $f(n) = \omega(g(n))$.

The easiest approach is the limit test, showing $\lim_{n \rightarrow \infty} \frac{a^n}{b^n} = \infty$. We can rewrite $\frac{a^n}{b^n} = \left(\frac{a}{b}\right)^n$.

Since $a > b > 1$, we know $\frac{a}{b} > 1$, and any value > 1 raised to increasing powers will itself continually increase; that is, the limit goes to infinity.

Student who want to use the formal definition of ω will need to find n_0 in terms of c , since the bound must work for any positive c .

(4) Given a set of n real numbers, the range of those numbers is the difference between the largest and smallest values. Let the numbers be stored in the various data structures listed below. For each one, describe in English an efficient algorithm for calculating the range and give the time complexity in asymptotic notation.

- (a) An unsorted array
- (b) A sorted array
- (c) A sorted singly-linked list
- (d) A balanced binary search tree

(a) There are no options besides examining every element of the array. This requires $\Theta(n)$ time.

(b) Since an array is random-access, we can just look at the first and last element and calculate the range in $\Theta(1)$ time.

(c) We again want the first and last elements, but we can't get the last element without walking down the entire linked list, so it's $\Theta(n)$ time.

(d) Recall that the smallest element will be all the way to the left and the largest will be all the way to the right. Since the tree is balanced, it has at most $\lceil \lg n \rceil$ levels, and we'll need that many steps to get the min and max values. So the entire process is $\Theta(\log n)$.

(5) Consider the following pseudocode:

```
# Input: A nonnegative integer
def func1(n):
    S = 0
    for i = 1 to n:
        S = S + i*i
    return S
```

- (a) What does this function compute?
- (b) What is the critical operation?
- (c) What is the efficiency class of this algorithm?
- (d) Either suggest an improvement or present an argument that this algorithm is optimal.

(a) The sum of the first n perfect squares

(b) The squaring operation ($i * i$)

(c) It's linear in n .

(d) No iterative process is likely to do better, but we know from FOCS that there is a closed-form solution to the summation of the first n perfect squares: $n(n + 1)(2n + 1)/6$. This would be a constant-time approach.

(6) Consider the following pseudocode for determining if a directed graph is symmetric (i.e. all edges can be traversed in both directions):

```
# Input: An nxn binary adjacency matrix
# note - vertices will be numbered 0 to n-1
def isSymmetric(A,n):
    if n = 1 return True
    for j = 0 to n-2:
        if A[j,n-1] != A[n-1,j] return False
    return isSymmetric(A,n-1)
```

Perform a worst-case time complexity analysis, including setting up and solving a recurrence relation. What is the function's asymptotic complexity class?

The worst case occurs when the graph is symmetric. In this case, the number of steps $T(n) = T(n - 1) + (n - 1)$, since the loop executes $n - 1$ times. This produces the standard $1 + 2 + \dots + n - 1$ summation, which is quadratic.